

SPENGERGASSE  ausbildung mit zukunft	HTBLVA, Wien V, Spengergasse Aufbaulehrgang für Informatik Kolleg für Informatik	Reife- und Diplomprüfung
--	---	-------------------------------------

Aufgabenstellung für die nicht standardisierte Klausurprüfung

Sommertermin 2025/26

Jahrgang:	6AAIF, 6BAIF, 6CAIF, 6AKIF, 6BKIF, 8ABIF, 8ACIF
Prüfungsgebiet:	Fachtheorie
Zugeteilter Pflichtgegenstand (Zugeteilte Pflichtgegenstände)	Programmieren und Software Engineering
Prüfungstag:	6.05.2026
Arbeitszeit:	300 Minuten
Kandidaten/Kandidatinnen:	30
Prüfer/Prüferin:	Michael Schletz, BEd und René Wenz, BSc
Aufgabenblätter:	9 Blätter plus Umschlagbogen

(Unterschrift des Prüfers/der Prüferin auf den jeweiligen Aufgabenblättern)

Das versiegelte Kuvert mit der Aufgabenstellung wurde geöffnet von:

Name: _____ Unterschrift: _____

Datum: _____ Uhrzeit: _____

Zwei Zeugen (Kandidaten/Kandidatinnen)

Name: _____ Unterschrift: _____

Name: _____ Unterschrift: _____

Geprüft: Wien, am _____

 AV Mag. Heidi Steinwender
 Abteilungsvorständin

RS.

 Dir. Dr. Gerhard Hager
 Direktor

Genehmigt:

Wien, am _____

RS.

 Mag.^a MinRⁱⁿ Gabriele Winkler-Rigler
 Schulaufsicht

Klausurprüfung in Programmieren und Software Engineering

Klassen: 6xAIF, 6xKIF, 8ABIF, 8ACIF

Datum: MI, 6. Mai 2026

Arbeitszeit: 300 Minuten

Generelle Hinweise zur Bearbeitung

Die Arbeitszeit für die Bearbeitung der gestellten Aufgaben beträgt 5 Stunden (300 Minuten). Die 3 Teilaufgaben sind unabhängig voneinander zu bearbeiten, Sie können sich die Zeit frei einteilen. Wir empfehlen jedoch eine maximale Bearbeitungszeit von 2 Stunden für Aufgabe 1, 1.5 Stunden für Aufgabe 2 und 1.5 Stunden für Aufgabe 3. Für eine Einrechnung der Jahresnote sind mindestens 30% der Gesamtpunkte zu erreichen.

Hilfsmittel

Sie finden ein in Ihrem Technologiestack entsprechendes Projektsetup mit allen Dependencies auf dem Netzlaufwerk **P:/SPG_Fachtheorie** vor.

Im Labor steht Visual Studio mit der .NET Version 10 bzw. IntelliJ Ultimate mit zumindest JDK21 zur Verfügung. Da das Projekt auf dem Netzlaufwerk liegt, muss am Ende nicht extra auf ein Abgabelaufwerk kopiert werden. **Beenden** Sie am Ende Ihrer Arbeit **alle Programme** und lassen Sie den PC eingeschaltet. Als erlaubte Hilfsmittel befinden sich im Ordner **R:\exams** bereitgestellte Unterlagen.

Wichtiger Hinweis vor Arbeitsbeginn



Füllen Sie die Datei *README.md* in *SPG_Fachtheorie/README.md* mit Ihren Daten (Klasse, Name und Accountname) aus. Sie sehen die Datei in Visual Studio unter *Solution Items* nach dem Öffnen der Solution. **Falls Sie dies nicht machen, kann Ihre Arbeit nicht zugeordnet und daher nicht bewertet werden!**



Während der Prüfung ist der Internetzugriff gesperrt. Für Visual Studio: Führen Sie niemals *Build - Rebuild Solution* aus, denn dadurch werden die lokalen Pakete gelöscht. Arbeiten Sie immer mit *Build - Build Solution* (F6).

Teilaufgabe 1: Object Relation Mapping

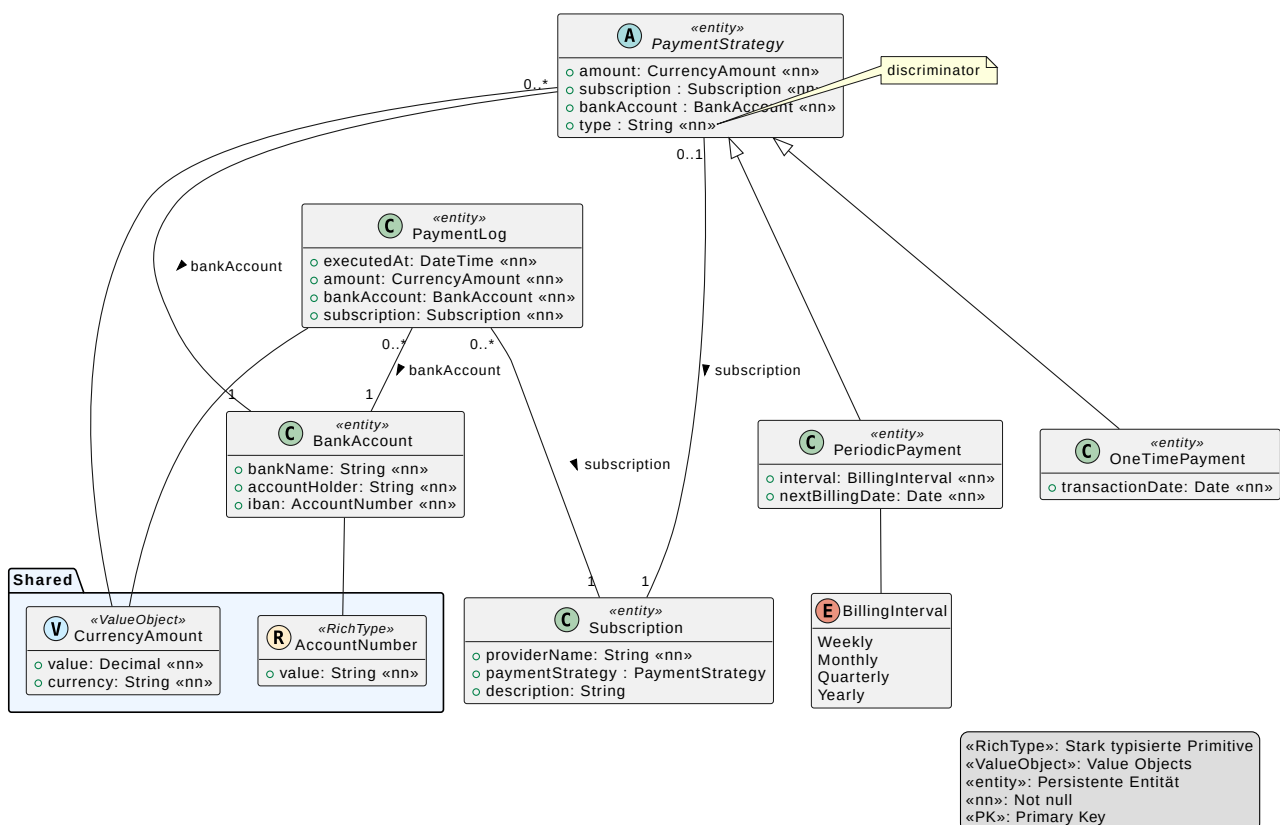
Immer mehr Services, die für unsere Arbeit nötig sind, werden in Form eines Abonnements erworben. Dies führt jedoch dazu, dass immer mehr Menschen den Überblick verlieren und mehr pro Monat ausgeben, als sie wirtschaftlich verkraften. Es soll daher eine App entwickelt werden, mit dessen Hilfe Zahlungen verwaltet werden können.

Zuerst wird ein Konto (*BankAccount*) mit den Feldern *bankName*, *accountHolder* (Inhaber) und *iban* angelegt. Wird ein neues Abonnement abgeschlossen, wird eine *Subscription* mit den Feldern *providerName* (Anbieter), *paymentStrategy* und *description* angelegt.

Die *PaymentStrategy* unterscheidet zwischen Einmalzahlungen (*OneTimePayment*), falls das Abonnement nicht automatisch verlängert wird, und *PeriodicPayment*, wenn das Abonnement automatisch verlängert wird.

Erfolgt eine Abbuchung, wird dies in *PaymentLog* vermerkt.

Das folgende Diagramm beschreibt das zu implementierende Domänenmodell:



Arbeitsauftrag

Erstellung der Modelklassen

Implementieren Sie das dargestellte Diagramm als persistierbare Modelklassen im Technologiestack ihrer Wahl.

C#/.NET

Im Projekt *SPG_Fachtheorie_Aufgabe1* befinden sich leere Klassen sowie die Klasse *AboManagerContext*, die Sie nutzen sollen.

Java

Implementieren Sie die im Modell dargestellten Domain-Klassen entsprechend. Benutzen Sie hierfür die vom Maven Build bereitgestellte H2 in-memory DB.

Alle Plattformen

Beachten Sie bei der Umsetzung, egal welche Plattform, die folgenden Punkte:

- Legen Sie sinnvolle *Konstruktoren* an:
 - Ein *public*-Konstruktor initialisiert alle Properties.
 - Ergänzen Sie die für das jeweilige Persistence Framework nötigen *protected* Konstruktoren.
- Die Tabellennamen sind in Einzahl in der Datenbank zu generieren (*PaymentLog*, etc.)
- *Required/Optional* gemäß Diagramm (<<nn>> = required).
- Zwischen *Subscription* und *PaymentStrategy* liegt eine 1 : 0..1 Beziehung vor. Bilden Sie diese korrekt über den OR Mapper ab.
- Das Feld *PaymentStrategy.type* wird als Discriminator verwendet. Er soll den Wert *P* für *PeriodicPayment* und *O* für *OneTimePayment* besitzen.
- *PaymentStrategy.Amount* ist ein *value object* und soll als solches konfiguriert werden.
- *PaymentLog.Amount* ist ein *value object* und soll als solches konfiguriert werden.
- Die Enum in *PeriodicPayment.Interval* soll als String gespeichert werden.
- *BankAccount.Iban* ist ein rich type. Er soll als String in der Datenbank gespeichert, und im Objektmodell als *AccountNumber* geladen werden.
- Das Feld *BankAccount.Iban* soll einen *unique index* besitzen. Stellen Sie durch Konfiguration sicher, dass nicht 2 Datensätze mit gleicher IBAN Nummer gespeichert werden können.

Verfassen von Tests

In der Klasse *Aufgabe1Test* im Projekt *test/SPG_Fachtheorie.Aufgabe1.Test* sollen Testmethoden verfasst werden, die die Richtigkeit der Konfiguration des OR Mappers beweisen sollen.

C#/.NET

- *T01_AddPaymentLogTest* weist nach, dass ein Eintrag in *PaymentLog* samt der in Beziehung stehenden Daten gespeichert werden kann.
- *T02_PaymentStrategyTypeHasCorrentValue* weist nach, dass der Discriminator in *PaymentStrategy.type* die richtigen Werte (*P* und *O*) besitzt.
- *T03_BankAccountIbanIsUniqueTest* weist nach, dass nicht zwei Datensätze mit gleicher IBAN Nummer in *BankAccount* gespeichert werden können.

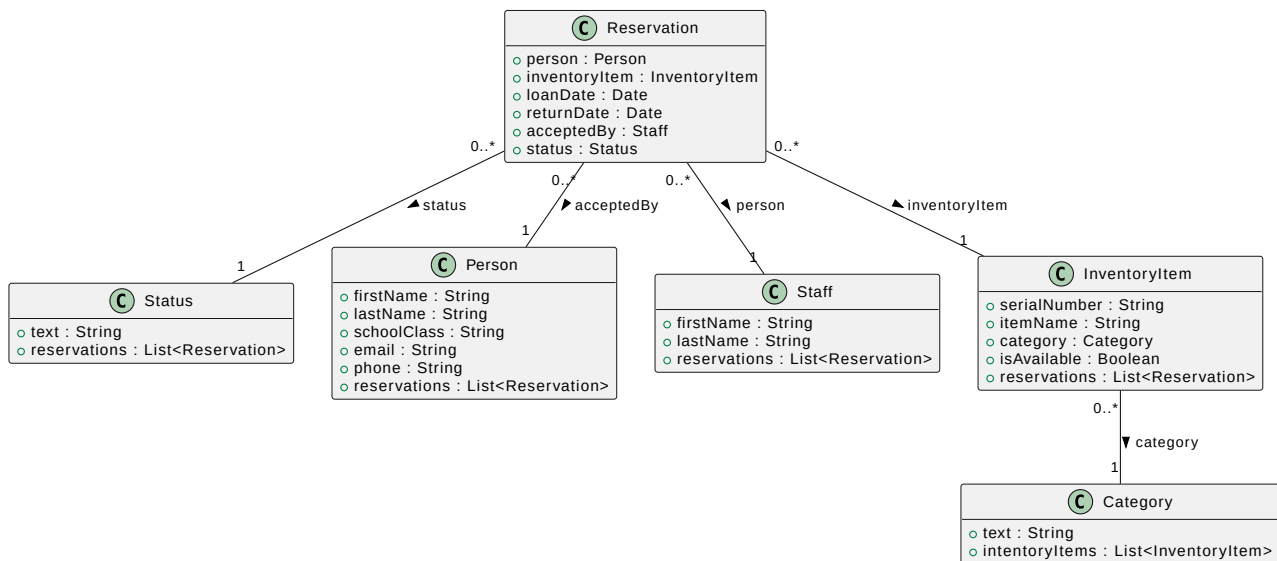
Sie können die vorgegebenen Tests in *test/SPG_Fachtheorie.Aufgabe1.Test/Aufgabe1MasterTests* verwenden, um Ihre Konfiguration bei der Bearbeitung der Aufgabenstellung zu prüfen.

Java

- Stellen Sie Converter-Tests zur Verfügung, die das korrekte Mapping sicherstellen.
- Stellen Sie notwendige Repository-Tests zur Verfügung, mit denen Sie die korrekte Persistierung des Modells sicherstellen. **HINWEIS:** Ein richtig geschriebener *AboManagerRepositoryTest* kann die Persistierung des gesamten Modells sicherstellen. Erweitern Sie diesen inkrementell.

Teilaufgabe 2: Abfragen und Servicemethoden

Das folgende Modell beschreibt ein vereinfachtes System für ein Entleihsystem der Schule. SchülerInnen und Lehrende (*Person*) können sich Gegenstände (*InventoryItem*) direkt im ZID ausleihen. Dafür wird ein Datensatz in *Reservation* angelegt. Die Werte *returnDate* und *acceptedBy* werden bei der Rückgabe gesetzt und sind vorher *null*. Es wird auch gespeichert, welche/r MitarbeiterIn (*Staff*) das Gerät zurücknimmt. Der Status ist eine Lookup Tabelle mit den Werten *Pending* (Rückgabe ausständig), *Issued* (ausgegeben) und *Damaged return* (beschädigt zurückgegeben).



Arbeitsauftrag

C#/.NET

In der Datei `src/SPG_Fachtheorie.Aufgabe2.Base/Infrastructure/ReservationContext.cs` steht ein vollständig implementierter DbContext zur Verfügung. Das Model ist bereits in `src/SPG_Fachtheorie.Aufgabe2.Base/Model` implementiert. **Sie müssen keine Implementierung des Modelles vornehmen, sondern es in Ihren Servicemethoden verwenden.** Führen Sie Ihre Implementierungen in `src/SPG_Fachtheorie.Aufgabe2/Services/ReservationService.cs` durch. Verwenden Sie den über Dependency Injection bereitgestellten *ReservationContext*.

Java

Es stehen Ihnen Repository Interfaces, Test-Fixtures und Test-Klassen zur Verfügung, um die Richtigkeit der Implementierung zu überprüfen. Erweitern Sie diese bei Bedarf. **Sie müssen keine Implementierung des Modelles oder Persistence-Mapping vornehmen, sondern die bereitgestellten Ressourcen in Ihren Servicemethoden verwenden.**

Implementierung der Servicemethoden

List<CategoryWithCount> getCategoriesWithCount()

Geben Sie alle Kategorien (*Category*) sowie die Anzahl der darin enthaltenen *InventoryItem* Datensätze zurück. Als Rückgabetytpe steht Ihnen der folgende Record zur Verfügung.

```
public record CategoryWithCount(string Text, int Count);
```

List<Person> getPersonsWithOpenReservations()

Geben Sie alle Personen zurück, die (mindestens) eine offene Reservierung besitzen. Sie erkennen offene Reservierungen daran, dass das Feld *returnDate* den Wert *null* hat.

List<InventoryItem> getInventoryItemsWithoutReservations(int year)

Geben Sie alle Gegenstände (*InventoryItem*) zurück, die im übergebenen Jahr *nicht* ausgeliehen wurden, d. h. für die es keinen Datensatz in *Reservation* für dieses Jahr gibt.

Reservation addReservation(int personId, int inventoryItemId, DateOnly loanDate)

Diese Methode soll eine Reservierung (*Reservation*) anlegen. Dabei sind folgende Randbedingungen zu prüfen:

- Ist der Parameter *personId* ungültig, d. h. es gibt keine entsprechende Person in der Datenbank, werfen Sie eine *ReservationException* mit dem Text *Invalid person..*
- Ist der Parameter *inventoryItemId* ungültig, d. h. es gibt kein entsprechendes *InventoryItem* in der Datenbank, werfen Sie eine *ReservationException* mit dem Text *Invalid inventory item..*
- Ist der Wert in *InventoryItem.isAvailable* auf *false* gesetzt, werfen Sie eine *ReservationException* mit dem Text *Item not available..*

Werden alle Bedingungen erfüllt, sollen Sie ein neues *Reservation* Objekt anlegen und in der Datenbank speichern. Verwenden Sie den Status mit dem Text *Pending* beim Erstellen der Reservierung. Die Werte *returnDate* und *acceptedBy* sind natürlich *null*.

Testen Ihrer Implementierung

Im Musterprojekt befinden sich in *SPG_Fachtheorie.Aufgabe2.Test/Aufgabe2MasterTests* vordefinierte Tests, die Ihre Implementierung testen. Sie müssen keine eigenen Unittests verfassen.

Teilaufgabe 3: REST(ful) API

Für das vorige Modell des Verleihsystems soll eine RESTful API implementiert werden.

Arbeitsauftrag

Implementieren Sie die folgenden REST API Routen im Presentation Package des von Ihnen gewählten Technologie-Stacks von *src/Aufgabe3*. Wie in Aufgabe 2 stehen Ihnen Test-Fixtures und ein Service-Interface für Mocking in den Tests zur Verfügung.

C#/.NET

In *src/SPG_Fachtheorie.Aufgabe3/Controllers/InventoryItemsController.cs* steht dafür ein Controller mit leeren Methoden bereit. Wenn Sie das Projekt in *src/SPG_Fachtheorie.Aufgabe3* starten, steht Ihnen unter der URL <http://localhost:5080/swagger/index.html> ein Endpoint Explorer zur Verfügung. Über Dependency Injection wird ein mit Musterdaten gefüllter *ReservationContext* bereitgestellt. Die Datenbank beinhaltet Musterdaten, sodass Sie die Funktionalität Ihrer Controller damit testen können. Sie können die Abfragen direkt in den Controllermethoden implementieren. Führen Sie alle Abfragen nicht blockierend (mit *await* und *async*) durch. Achten Sie bei Fehlern auf eine RFC-9457 kompatible Antwort, indem Sie die Methode *Problem()* mit geeignetem Statuscode verwenden.

Java

Verwenden Sie die vom Setup bereitgestellte *ProblemDetail* (RFC-9457) Implementierung. Sie sollen weder Service noch Domain-Modell bzw. Persistence-Mapping in dieser Aufgabenstellung implementieren.

GET /inventoryItems?withReservations=(true | false)

Diese REST API Route soll eine Liste aller Gegenstände (*InventoryItem*) zurückgeben. Ist der Query Parameter *withReservations* auf *true* gesetzt, so ist im Datensatz die Liste der Reservierungen dieses Gegenstandes zurückzugeben. Ist der Parameter auf *false* gesetzt, ist ein leeres Array (*[]*) bei *reservations* auszugeben. Für die Rückgabe steht ihnen folgender Record zur Verfügung:

```
public record InventoryItemWithReservationsDto(  
    int Id, string SerialNumber, string ItemName,  
    List<ReservationDto> Reservations);  
  
public record ReservationDto(  
    int Id, DateOnly LoanDate, DateOnly? ReturnDate,  
    string PersonFirstName, string PersonLastName);
```

Erwartete HTTP-Antworten:

HTTP Status	Bedingung
200	Ein Array von <i>InventoryItemWithReservationsDto</i> Objekten mit allen Gegenständen und Reservierungen, je nach gesetztem Parameterwert.

GET /inventoryItems/{category}

Diese REST API Route soll die Liste aller Gegenstände (*InventoryItem*), die einer bestimmten Kategorie zugeordnet sind, zurückgeben. In der Datenbank befinden sich die Kategorien *Notebooks*, *Adapters* und *Cables*, die Sie zum Testen des Filters verwenden können.

Für die Rückgabe steht ihnen folgender Record zur Verfügung:

```
public record InventoryItemDto(int Id, string SerialNumber, string ItemName);
```

Erwartete HTTP-Antworten:

HTTP Status	Bedingung
200	Eine Liste von <i>InventoryItemDto</i> Objekten der entsprechenden Kategorie mit den erforderlichen Daten.

PATCH /inventoryItems/available/{id}

Diese REST API Route soll das Feld *isAvailable* in *InventoryItem* aktualisieren. Über den Routingparameter bekommen Sie die ID des *InventoryItem* Datensatzes. Als Command Object steht Ihnen folgender Record zur Verfügung:

```
public record UpdateAvailableCmd(bool isAvailable);
```

Wird die ID nicht in *InventoryItem* gefunden, sollen Sie HTTP 404 (not found) mit RFC problem detail body senden.

Erwartete HTTP-Antworten:

HTTP Status	Bedingung
204	Das Flag <i>IsAvailable</i> konnte aktualisiert werden.
404	Die ID wurde nicht gefunden.

Verfassen von Integration Tests

Es sollen Integration Tests **für den PATCH Endpunkt** verfasst werden. Es soll jede Zeile in den Tabellen bei *Erwartete HTTP-Antworten* geprüft werden.

C#/.NET

In `test/SPG_Fachtheorie.Aufgabe3.Test/InventoryItemsControllerTests.cs` sollen Integration Tests für den **PATCH Endpunkt** verfasst werden. Verwenden Sie dafür die in der `TestWebApplicationFactory` bereitgestellten Hilfsmethoden:

Methode	Beschreibung
InitializeDatabase	Erstellt eine leere Datenbank und fügt ggf. Werte ein.
QueryDatabase	Erlaubt das Abfragen der Datenbank.
GetHttpContent<T>	Führt einen GET Request durch und liefert das Ergebnis als Typ <i>T</i> .
PatchHttpContent	Führt einen PATCH Request durch und liefert den Status Code zurück.

Java

Setzen Sie die Tests mittels Mocking des Services bzw. Repositories in Ihrem Testframework um.

Bewertungsblatt

In Summe sind 74 Punkte zu erreichen. Für eine Berücksichtigung der Jahresnote müssen mindestens 30 % der Gesamtpunkte (also 23) erreicht werden. Für eine positive Beurteilung der Klausur müssen mindestens 50 % der Gesamtpunkte erreicht werden.

Beurteilungsstufen:

74 – 65 Punkte: Sehr gut, 64 – 56 Punkte: Gut, 55 – 47 Punkte: Befriedigend, 46 – 37 Punkte: Genügend

Aufgabe 1 (28 Punkte in Summe)	Ges	Pkt.
Das Entity PaymentStrategy wird korrekt in der erzeugten Datenbank abgebildet.	1	
Das Entity PeriodicPayment wird korrekt in der erzeugten Datenbank abgebildet.	1	
Das Entity OneTimePayment wird korrekt in der erzeugten Datenbank abgebildet.	1	
Das Entity PaymentLog wird korrekt in der erzeugten Datenbank abgebildet.	1	
Das Entity BankAccount wird korrekt in der erzeugten Datenbank abgebildet.	1	
Das Entity Subscription wird korrekt in der erzeugten Datenbank abgebildet.	1	
Die 1:0..1 Beziehung zwischen Subscription und PaymentStrategy wird korrekt in der Datenbank abgebildet.	2	
Das Feld PaymentStrategy.type wird als Discriminator verwendet.	1	
Der Discriminator hat die korrekten Werte (P und O).	1	
PaymentStrategy.Amount wird als value object konfiguriert.	2	
PaymentLog.Amount wird als value object konfiguriert.	2	
Die Enum Die Enum in PeriodicPayment.Interval wird als String gespeichert.	2	
BankAccount.Iban wird als rich type definiert und als String gespeichert.	2	
BankAccount.Iban wird mit einem unique index definiert.	1	
Der Test AddPaymentLogTest läuft durch und ist korrekt aufgebaut.	3	
Der Test PaymentStrategyTypeHasCorrentValue läuft durch und ist korrekt aufgebaut.	3	
Der Test BankAccountIbanIsUniqueTest läuft durch und ist korrekt aufgebaut.	3	
Aufgabe 2 (18 Punkte in Summe)	Ges	Pkt.
Die Methode getCategoriesWithCount ist korrekt.	3	
Die Methode getPersonsWithOpenReservations ist korrekt.	4	
Die Methode getInventoryItemsWithoutReservations ist korrekt.	4	
Die Methode addReservation prüft personId korrekt.	1	

Aufgabe 2 (18 Punkte in Summe)	Ges	Pkt.
Die Methode addReservation prüft personId korrekt.	1	
Die Methode addReservation prüft isAvailable korrekt.	1	
Die Methode addReservation erstellt eine korrekte Reservierung in der Datenbank.	4	

Aufgabe 3 (28 Punkte in Summe)	Ges	Pkt.
GET /inventoryItems liefert HTTP 200.	1	
GET /inventoryItems liefert die korrekten Daten bei withReservations=false	4	
GET /inventoryItems liefert die korrekten Daten bei withReservations=true	4	
GET /inventoryItems/{category} liefert HTTP 200.	1	
GET /inventoryItems liefert die korrekten Daten.	3	
PATCH /inventoryItems/available/{id} liefert HTTP 204 im Erfolgsfall.	1	
PATCH /inventoryItems/available/{id} ändert die Daten in der Datenbank korrekt.	3	
PATCH /inventoryItems/available/{id} liefert 404, wenn die ID nicht gefunden wurde.	2	
Der Integration Test für PATCH prüft HTTP 204 und läuft durch.	3	
Der Integration Test für PATCH prüft die Daten der Datenbank und läuft durch.	3	
Der Integration Test für PATCH prüft HTTP 404 und läuft durch.	3	